

Estructuras de Datos

Clase 5 – Implementación de Pilas y Colas con Nodos Enlazados



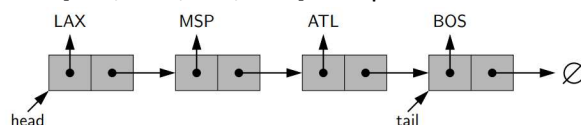
Dr. Sergio A. Gómez
<http://cs.uns.edu.ar/~sag>



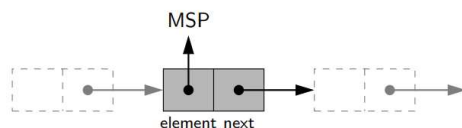
Departamento de Ciencias e Ingeniería de la Computación
 Universidad Nacional del Sur
 Bahía Blanca, Argentina

Secuencias con nodos enlazados

- La secuencia [LAX, MSP, ATL, BOS] se representa con:



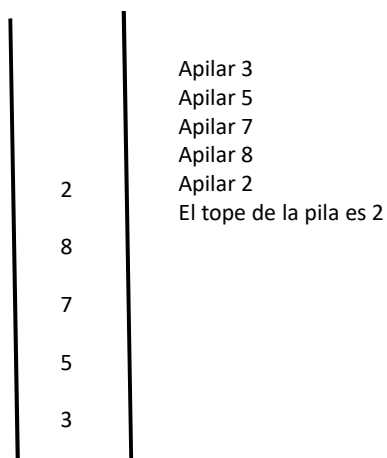
- Hay 2 campos *head* (para el primer elemento) y *tail* (para el último).
- Cada nodo conoce su elemento o rótulo (*element*) y el nodo siguiente en la secuencia (*next*) y se representa como:



- El símbolo $\emptyset$ indica que el siguiente es ninguno y, en Java, se representa con *null*.

Repaso: Tipo de dato abstracto Pila

Pila: Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).



Estructuras de datos - Dr. Sergio A. Gómez

3

Repaso: Interfaz para el TDA Pila

```
public interface Stack<E> {
    // Inserta item en el tope de la pila.
    public void push( E item );

    // Retorna true si la pila está vacía y falso en caso contrario.
    public boolean isEmpty();

    // Elimina el elemento del tope de la pila y lo retorna.
    // Produce un error si la pila está vacía.
    public E pop();

    // Retorna el elemento del tope de la pila y lo retorna.
    // Produce un error si la pila está vacía.
    public E top();

    // Retorna la cantidad de elementos de la pila.
    public int size();
}
```

Estructuras de datos - Dr. Sergio A. Gómez

4

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

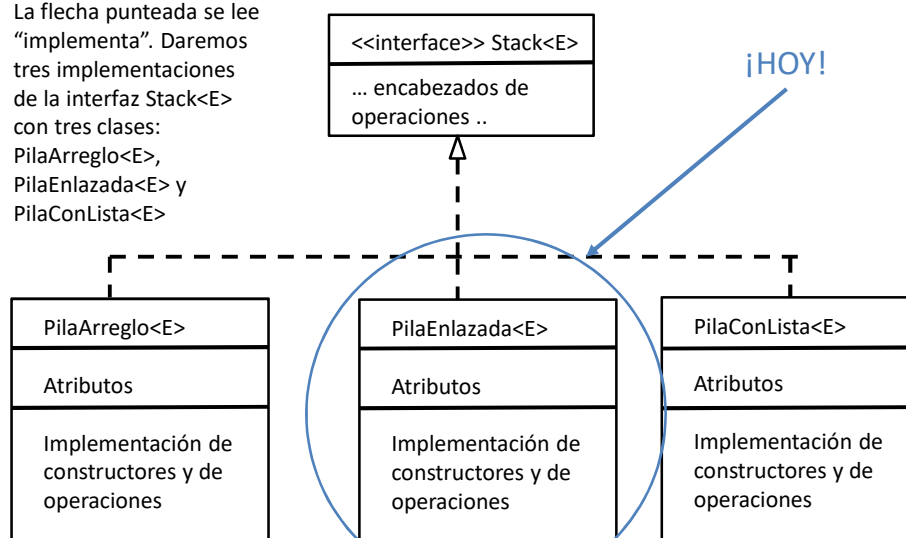
Implementaciones de pilas

- 1) Con un arreglo (en clase 3):
 - 1) Fácil de implementar y testear
 - 2) Es necesario indicar el tamaño máximo en el constructor
 - 3) La ED es acotada y genera problemas cuando se llena el arreglo para aumentar su tamaño
- 2) Con una estructura de nodos enlazados (hoy):
 - 1) Un poco más difícil de implementar y testear
 - 2) No es necesario indicar el tamaño en el constructor
 - 3) La estructura es no acotada, i.e. puede crecer indefinidamente (mientras haya memoria)
- 3) Con un adaptador en términos de lista (cuando demos el TDA Lista)

5

Diagrama UML del diseño

La flecha punteada se lee "implementa". Daremos tres implementaciones de la interfaz Stack<E> con tres clases: PilaArreglo<E>, PilaEnlazada<E> y PilaConLista<E>

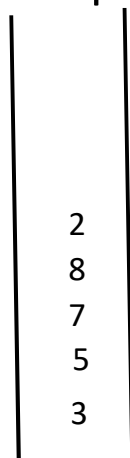


Estructuras de datos - Dr. Sergio A. Gómez

6

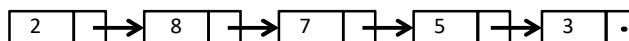
El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Implementación con nodos enlazados



s

Utilizaremos una estructura de nodos enlazados. La pila mostrada se representará con nodos (celdas) enlazados:



tope
(top)

Notas:

La pila conoce sólo el nodo *cabeza/tope*, que corresponde al tope o último dato apilado.

Todas las operaciones son eficientes pues la estructura es no acotada y no hay casos especiales porque nunca se llena.

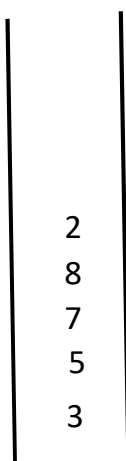
Es más difícil de codificar, depurar y mantener pero se gana en flexibilidad.

Se puede almacenar el tamaño como un atributo más para evitar recorrer la estructura para computar `size()`.

Estructuras de datos - Dr. Sergio A. Gómez

7

Pila: Implementación con nodos enlazados



s

// Archivo: PilaEnlazada.java

```
public class PilaEnlazada<E> implements Stack<E> {
    ...
}
```

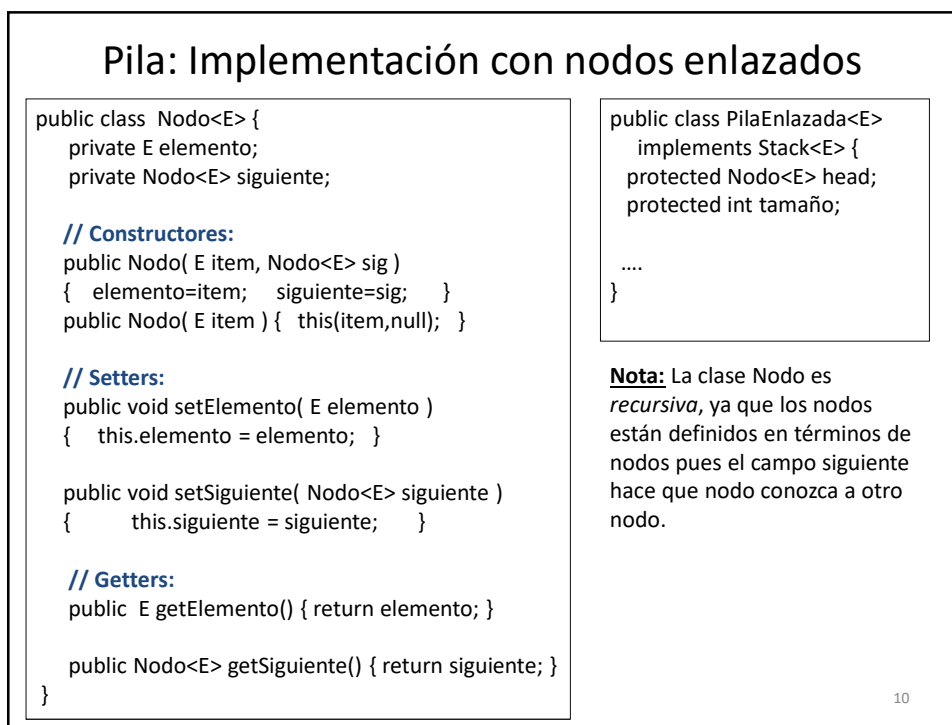
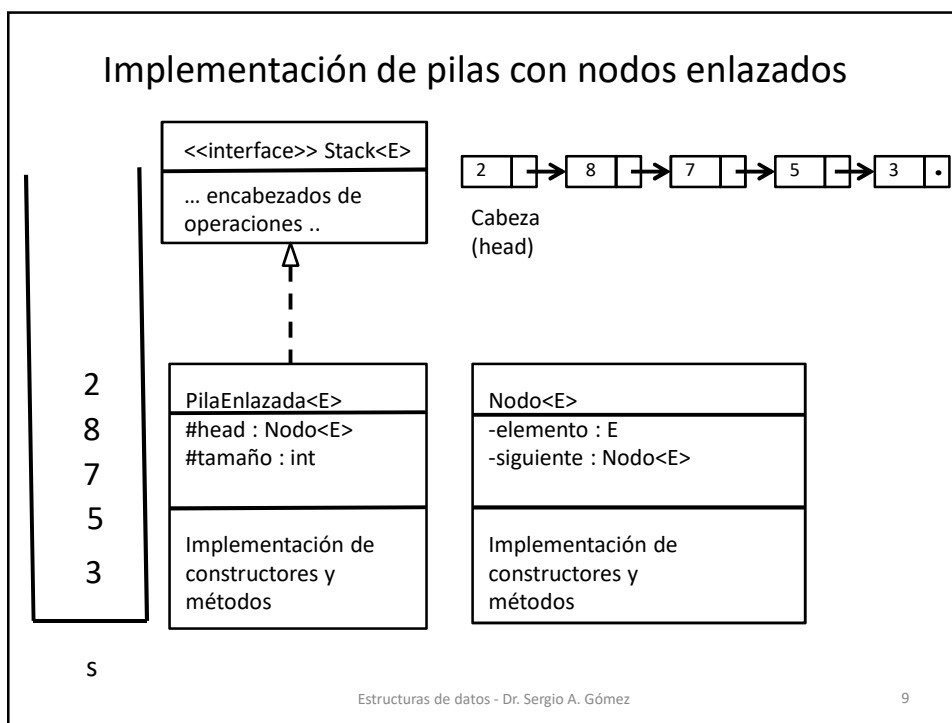
// Archivo: AplicacionUsaPilaEnlazada.java

```
public class AplicacionUsaPilaEnlazada {
    public static void main( String [] args ) {
        Stack<Integer> s = new PilaEnlazada<Integer>();
        s.push( 3 );
        s.push( 5 );
        s.push( 7 );
        s.push( 8 );
        s.push( 2 );
    }
}
```

Ahora la aplicación debe invocar el constructor de la nueva clase pero como la variable s es de tipo Stack el resto de la implementación de main no cambia.

8

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.



Pila con nodos enlazados: Constructor

El constructor crea una pila vacía.

Recordemos que:

```
Public class PilaEnlazada<E>
    implements Stack<E> {
    protected Nodo<E> head;
    protected int tamaño;

    ....
}
```

Constructor:

```
PilaEnlazada() {
    head = null
    tamaño = 0
}
```

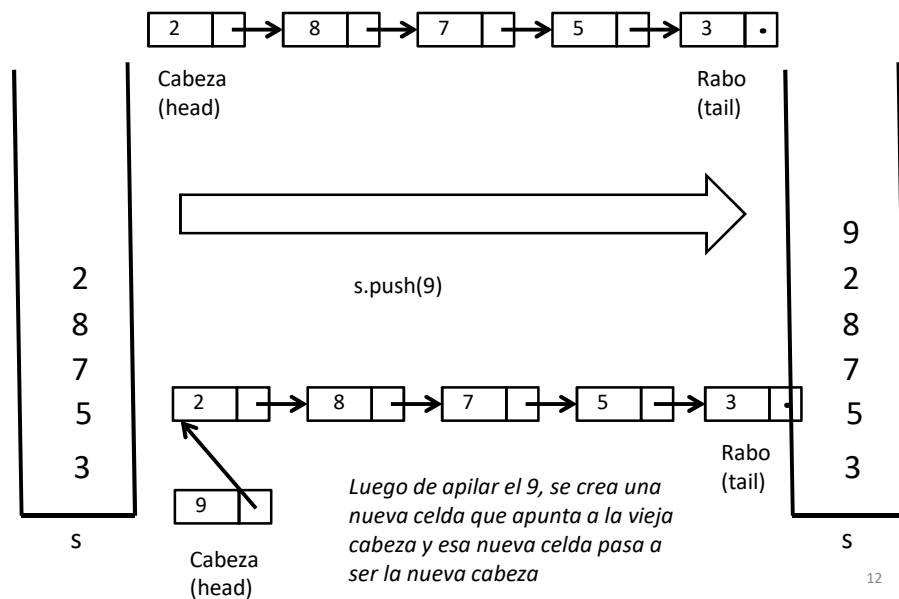
Un caso de prueba para validar el comportamiento del constructor y la operación isEmpty puede ser:

```
assert (new PilaEnlazada<E>()).isEmpty() == true;
```

S

11

Apilar



El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

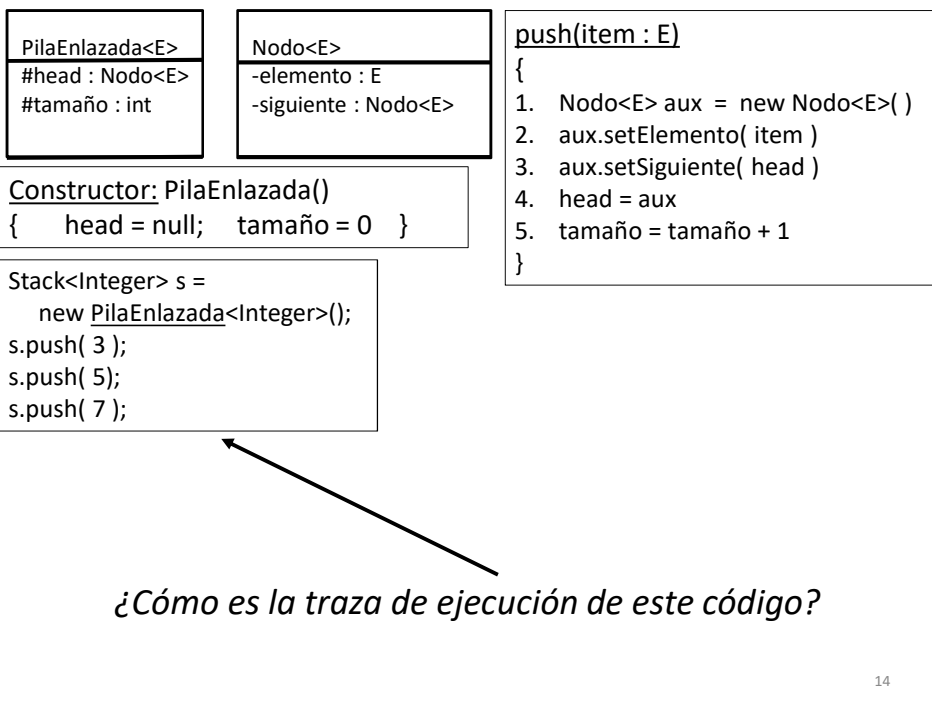
Apilar

```
Public class PilaEnlazada<E>
    implements Stack<E> {
    protected Nodo<E> head;
    protected int tamaño;

    ....
}
```

```
push(item : E)
{
    Nodo<E> aux = new Nodo<E>(item )
    aux.setElemento( item )
    aux.setSiguiente( head )
    head = aux
    tamaño = tamaño + 1
}
```

13



14

PilaEnlazada<E> #head : Nodo<E> #tamaño : int	Nodo<E> -elemento : E -siguiente : Nodo<E>	push(item : E) <pre>{ 1. Nodo<E> aux = new Nodo<E>() 2. aux.setElemento(item) 3. aux.setSiguiente(head) 4. head = aux 5. tamaño = tamaño + 1 }</pre>
--	---	--

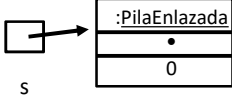
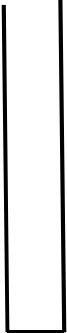
Constructor: PilaEnlazada()

```
{ head = null; tamaño = 0 }
```

Stack<Integer> s =
 new PilaEnlazada<Integer>();

Al ejecutar el constructor se crea un objeto de tipo PilaArreglo<Integer> y su referencia se almacena en s.

La pila conceptual es la que veremos a la derecha.

S
15

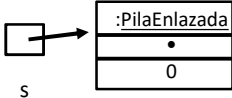
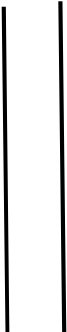
PilaEnlazada<E> #head : Nodo<E> #tamaño : int	Nodo<E> -elemento : E -siguiente : Nodo<E>	push(item : E) <pre>{ 1. Nodo<E> aux = new Nodo<E>() 2. aux.setElemento(item) 3. aux.setSiguiente(head) 4. head = aux 5. tamaño = tamaño + 1 }</pre>
--	---	--

Constructor: PilaEnlazada()

```
{ head = null; tamaño = 0 }
```

Stack<Integer> s =
 new PilaEnlazada<Integer>();
 s.push(3);

Ahora queremos apilar un 3

S
16

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

PilaEnlazada<E> #head : Nodo<E> #tamaño : int	Nodo<E> -elemento : E -siguiente : Nodo<E>	push(item : E) <pre>{ 1. Nodo<E> aux = new Nodo<E>() 2. aux.setElemento(item) 3. aux.setSiguiente(head) 4. head = aux 5. tamaño = tamaño + 1 }</pre>
Constructor: PilaEnlazada() <pre>{ head = null; tamaño = 0 }</pre>		

Stack<Integer> s =
 new PilaEnlazada<Integer>();
 s.push(3);

Ejecutamos push.1: se crea un nodo y se apunta con *aux*

S
17

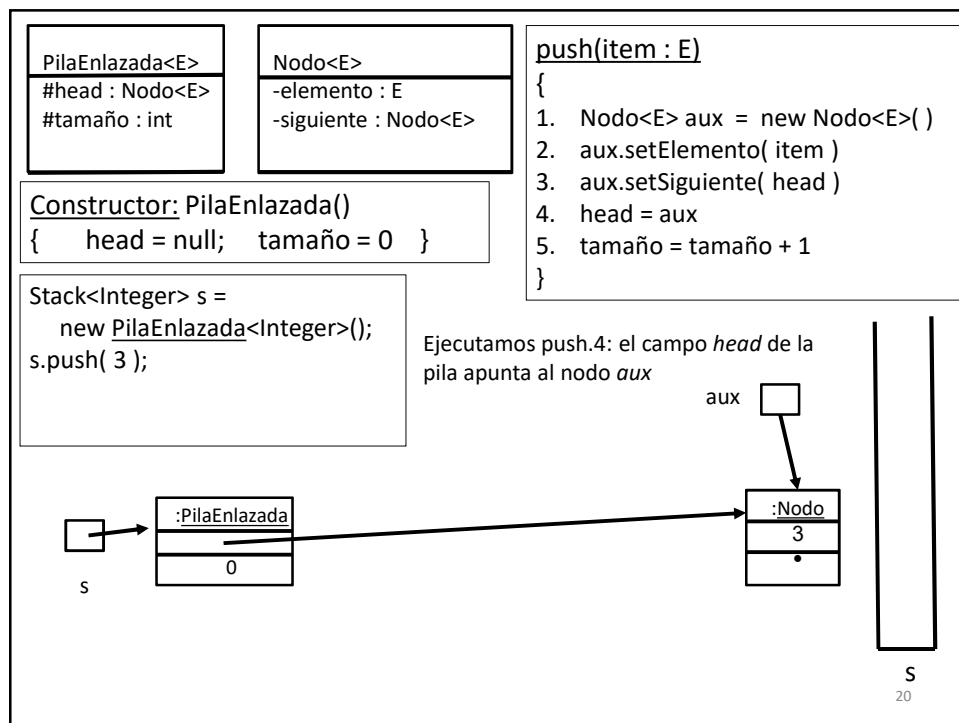
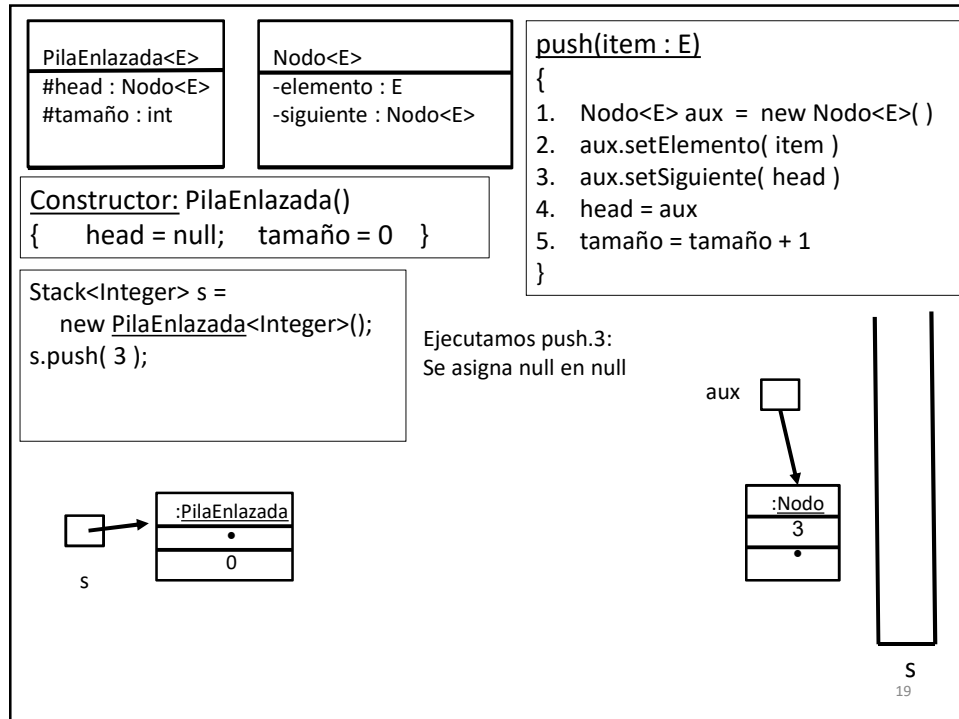
PilaEnlazada<E> #head : Nodo<E> #tamaño : int	Nodo<E> -elemento : E -siguiente : Nodo<E>	push(item : E) <pre>{ 1. Nodo<E> aux = new Nodo<E>() 2. aux.setElemento(item) 3. aux.setSiguiente(head) 4. head = aux 5. tamaño = tamaño + 1 }</pre>
Constructor: PilaEnlazada() <pre>{ head = null; tamaño = 0 }</pre>		

Stack<Integer> s =
 new PilaEnlazada<Integer>();
 s.push(3);

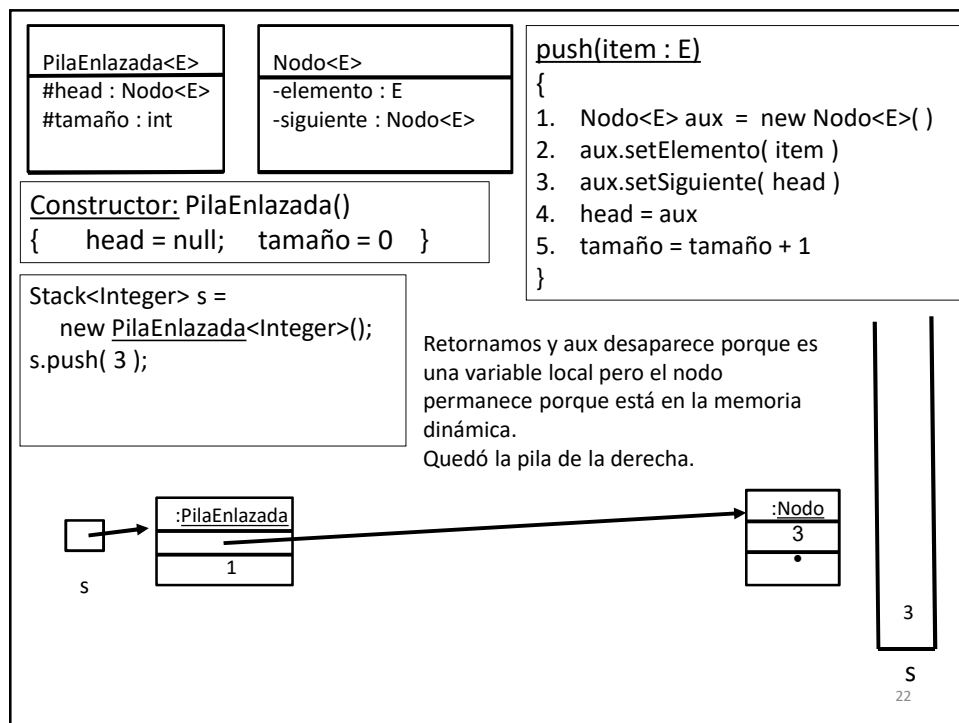
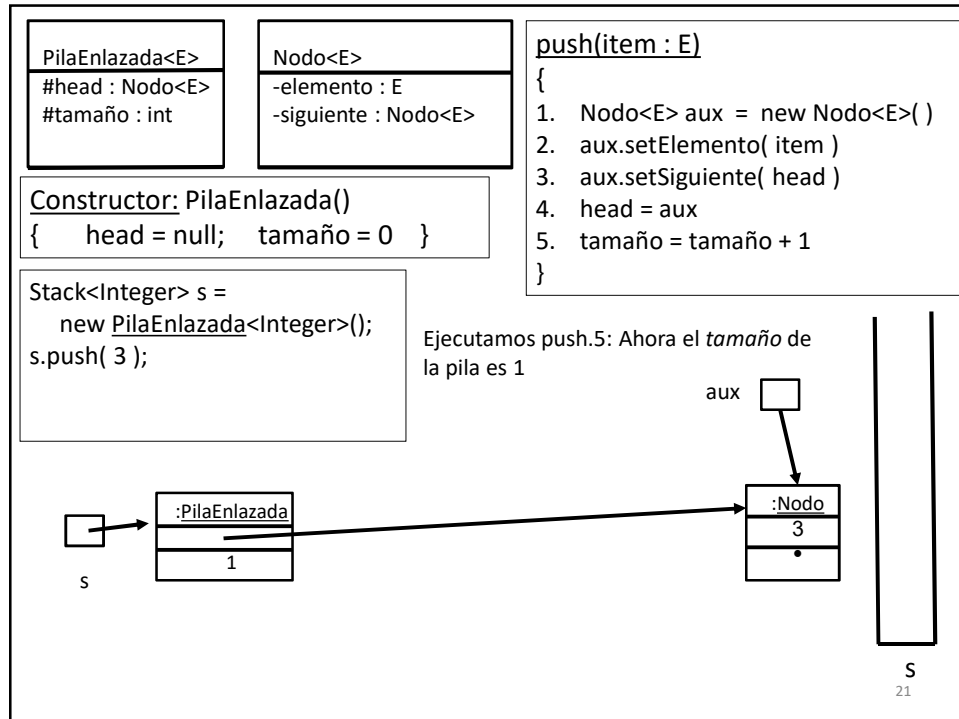
Ejecutamos push.2: asignamos 3 en campo *elemento* de *aux*

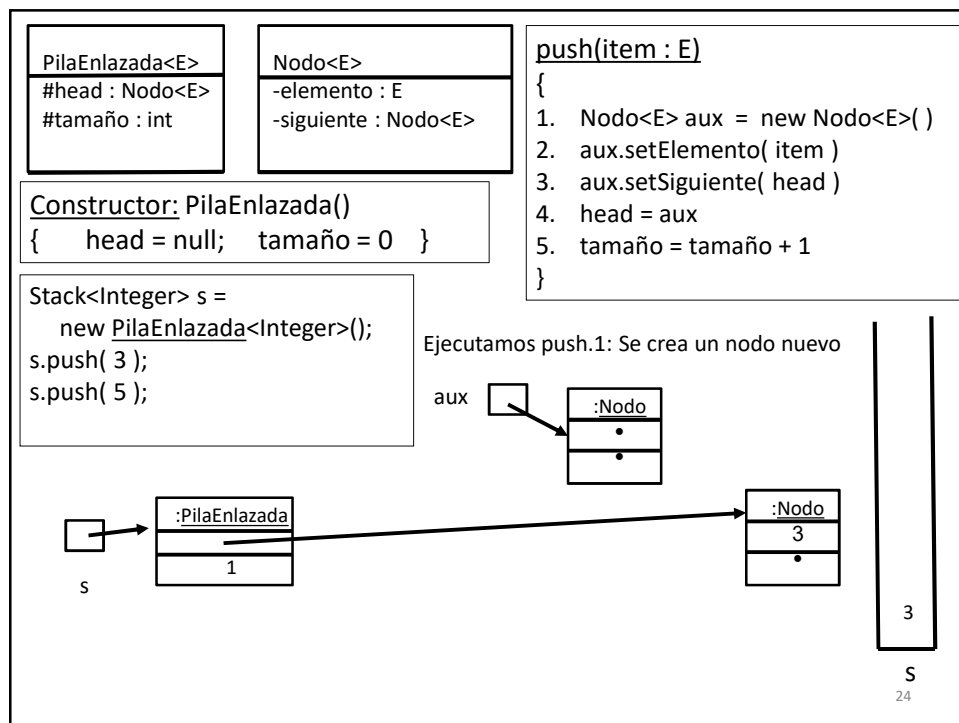
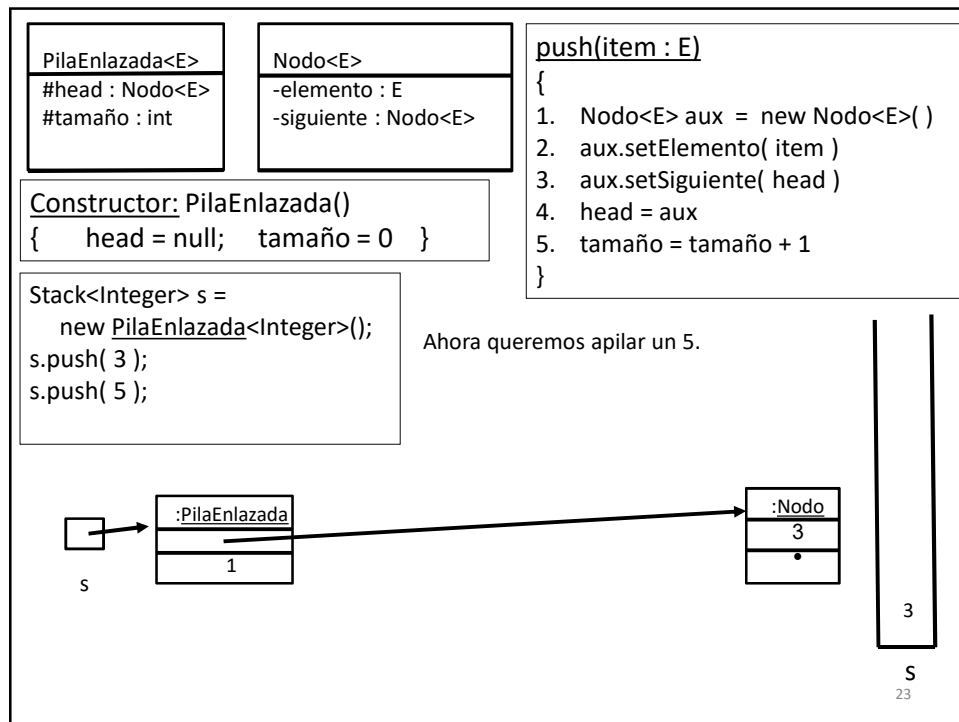
S
18

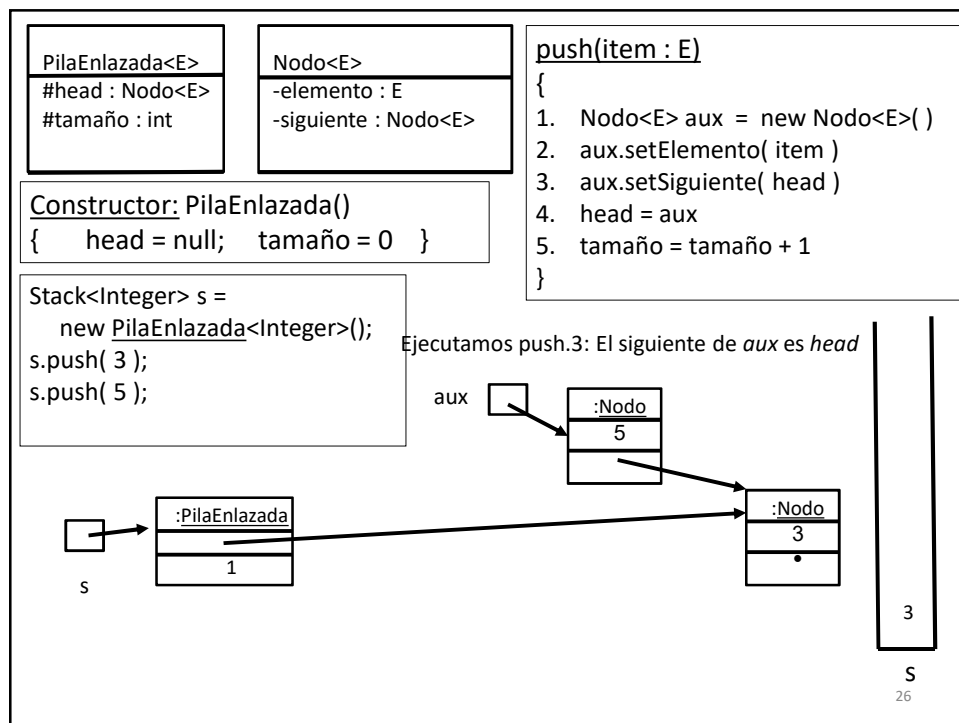
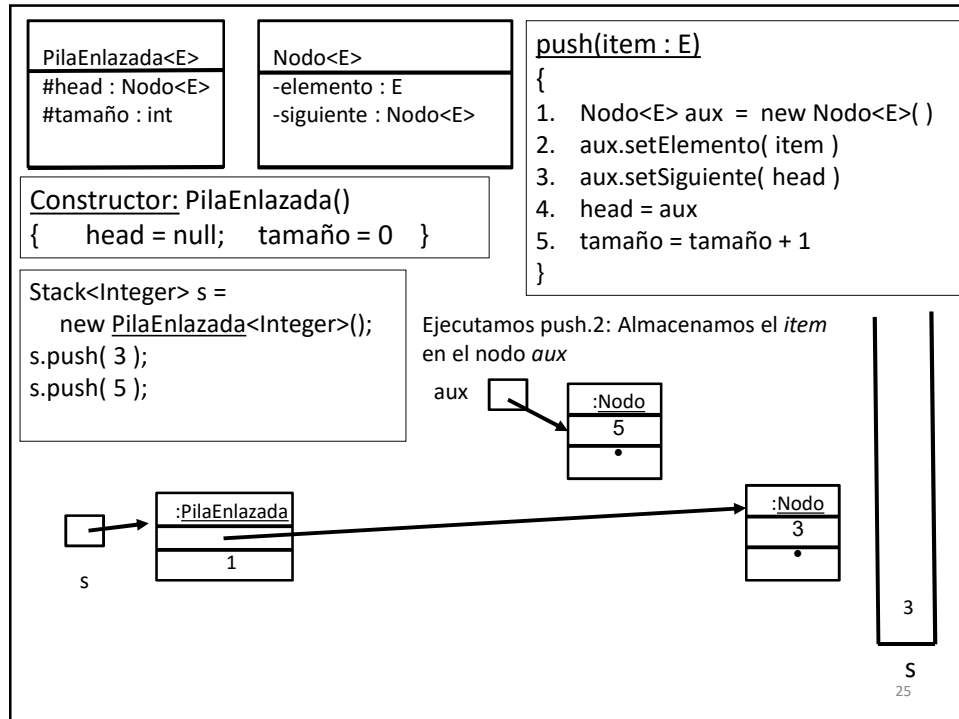
El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

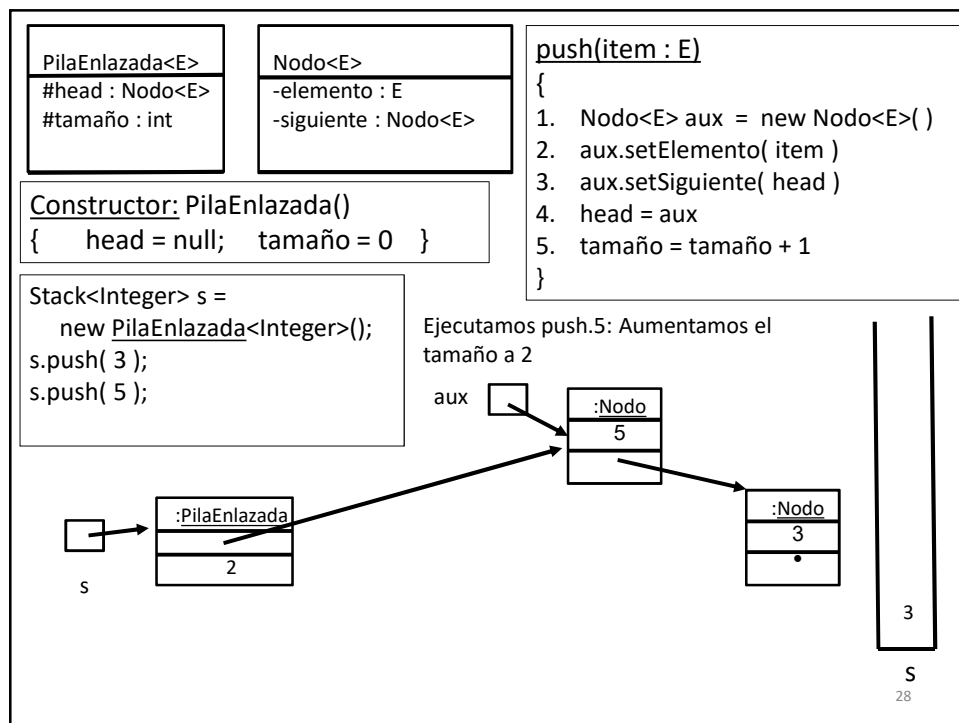
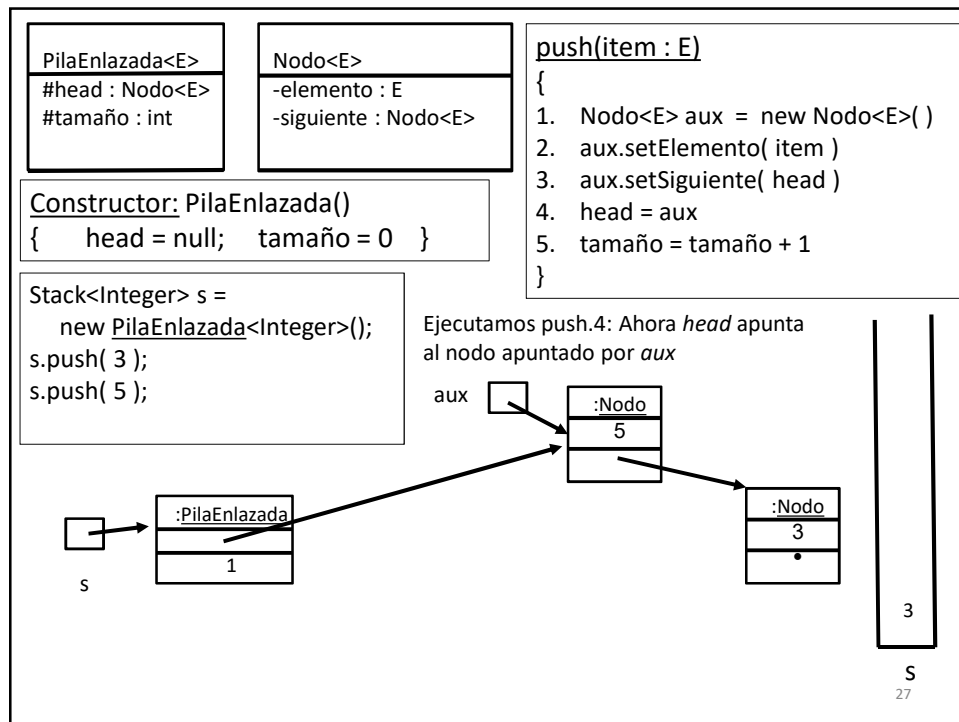


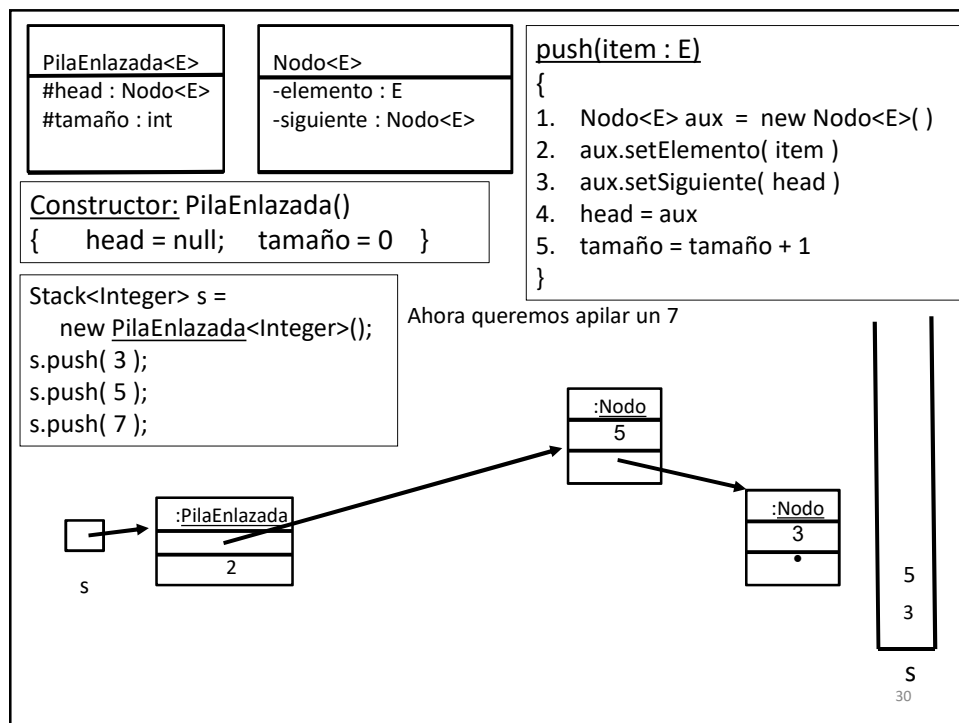
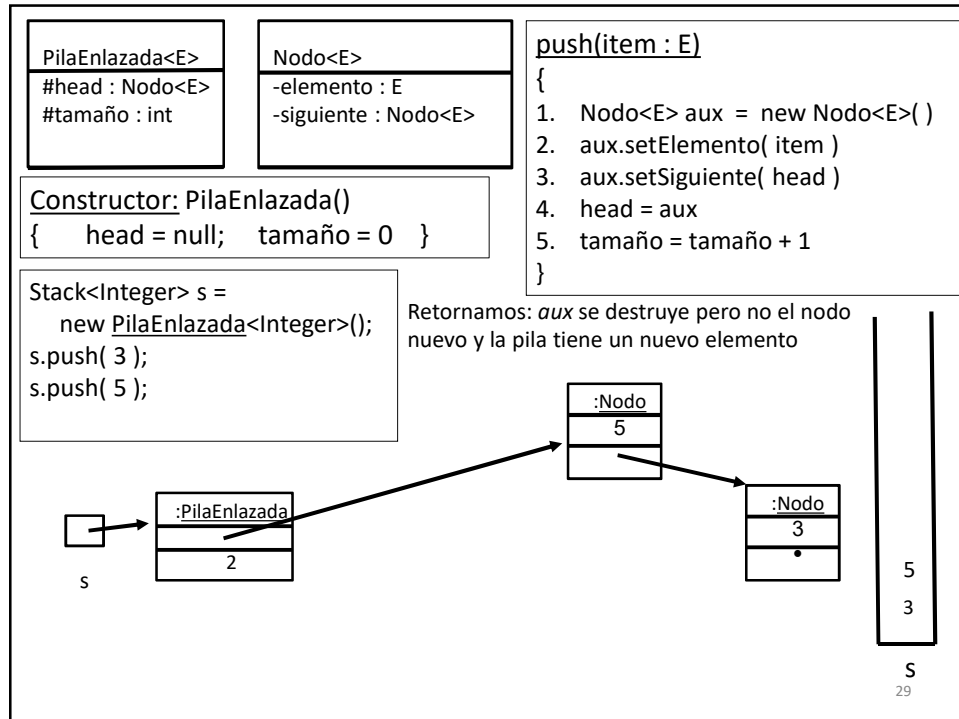
El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

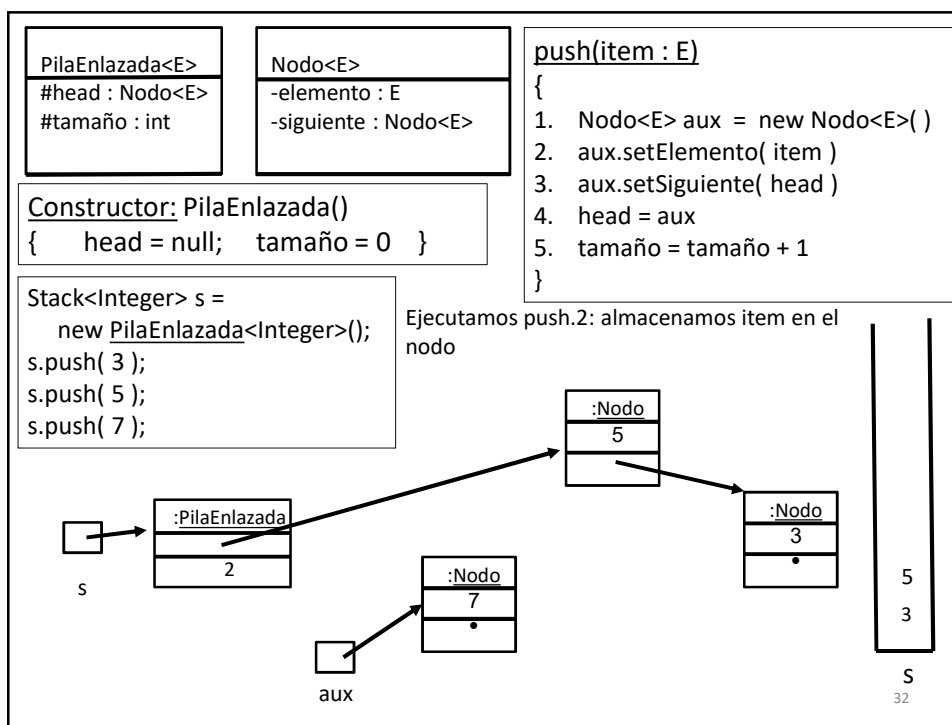
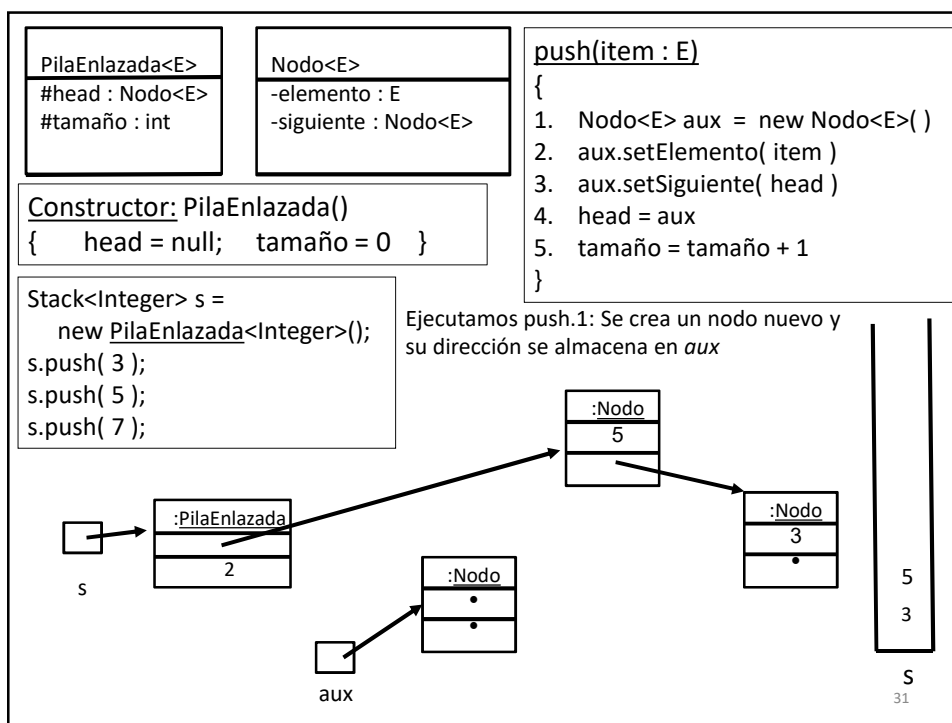




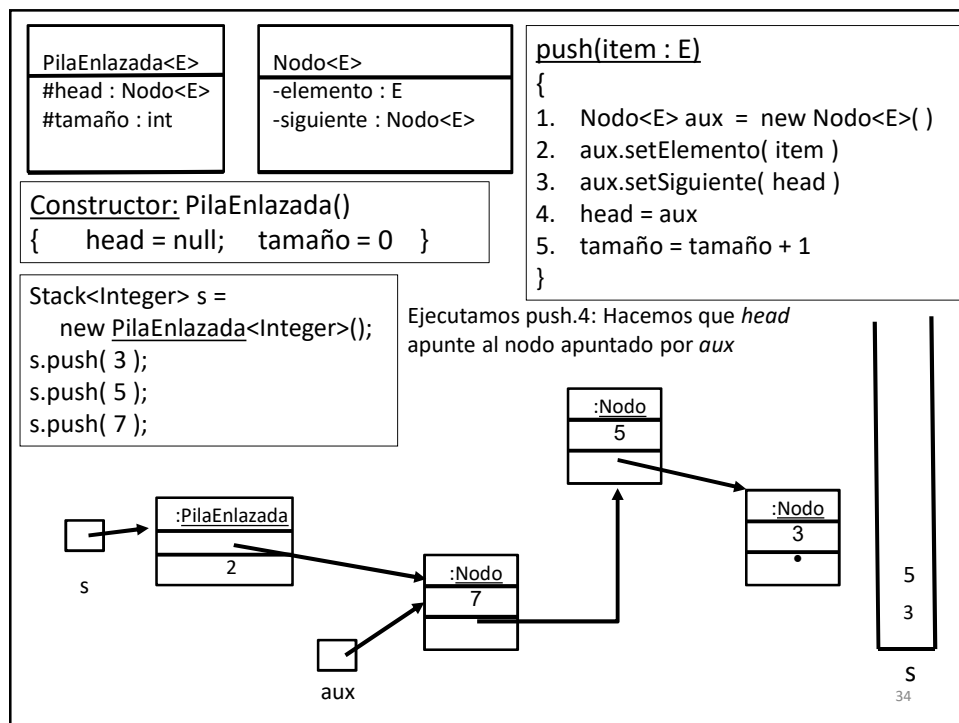
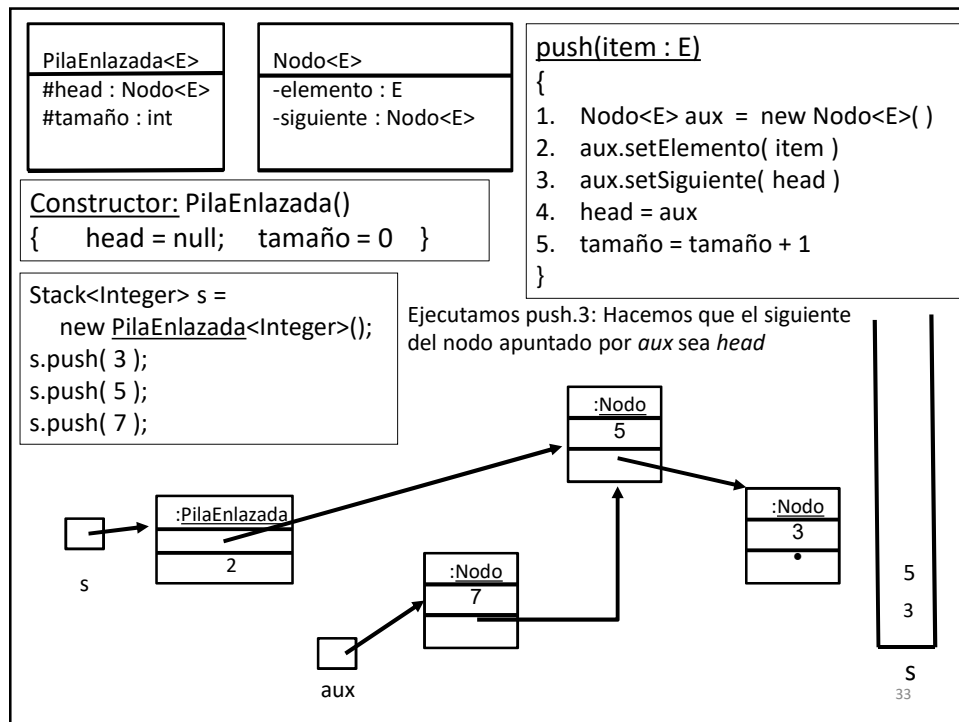




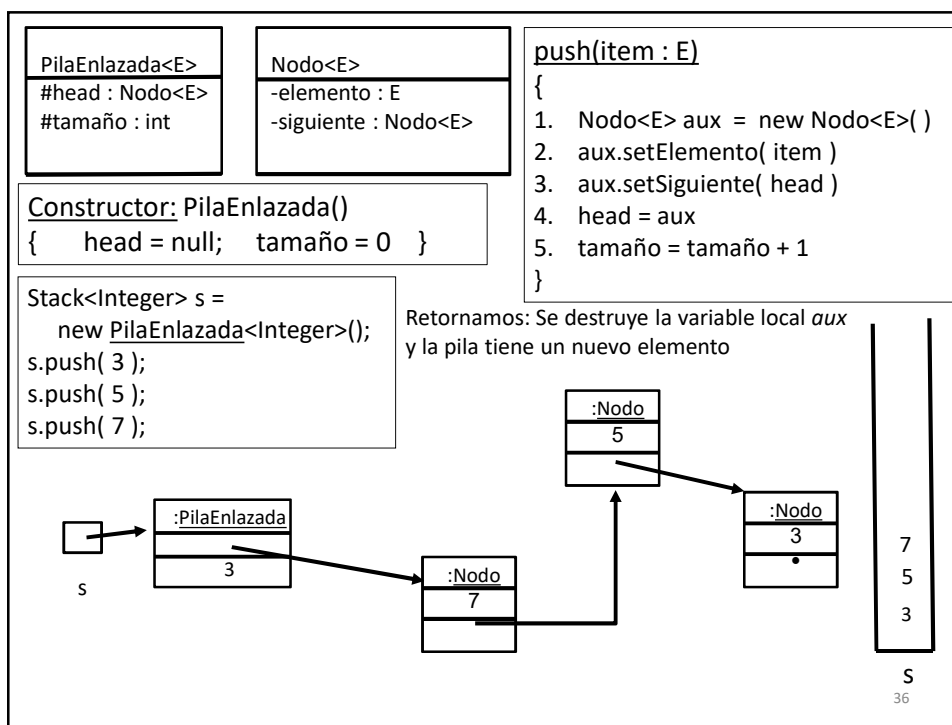
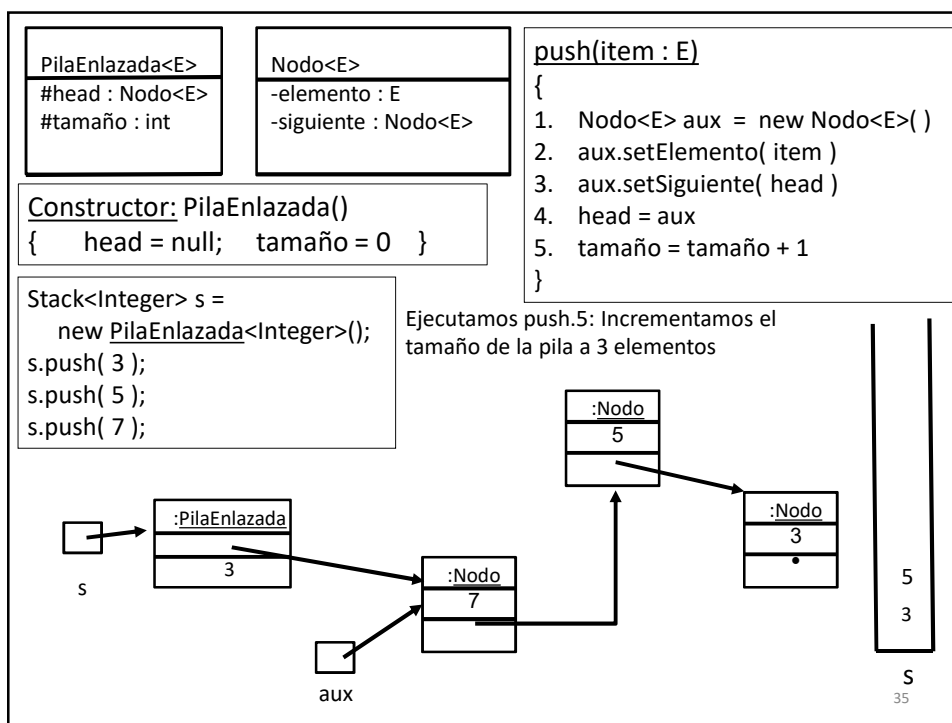




El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.



El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.



El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Para pensar: Variantes en la implementación de apilar

push(item : E)

```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

(1), (2) y (3) se pueden juntar

push(item : E)

```
{
  Nodo<E> aux = new Nodo<E>(item, head)
  head = aux
  tamaño = tamaño + 1
}
```

(1) y (2) se
pueden juntar

push(item : E)

```
{
  Nodo<E> aux = new Nodo<E>(item)
  aux.setSiguiente( head )
  head = aux
  tamaño = tamaño + 1
}
```

¡(1), (2), (3) y (4) se pueden juntar!

push(item : E)

```
{
  head = new Nodo<E>(item, head)
  tamaño++
}
```

Estructuras de datos – Dr. Sergio A. Gómez

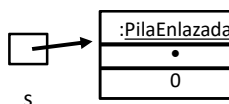
37

Operación isEmpty: boolean vacia = s.isEmpty()

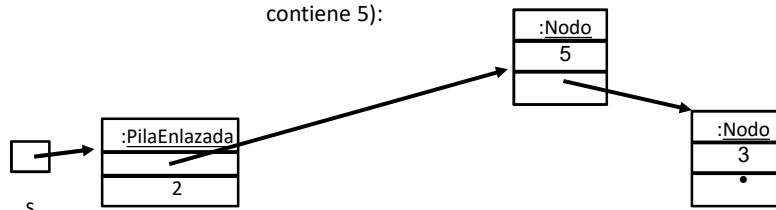
isEmpty() : boolean

```
{
  return head == null
}
```

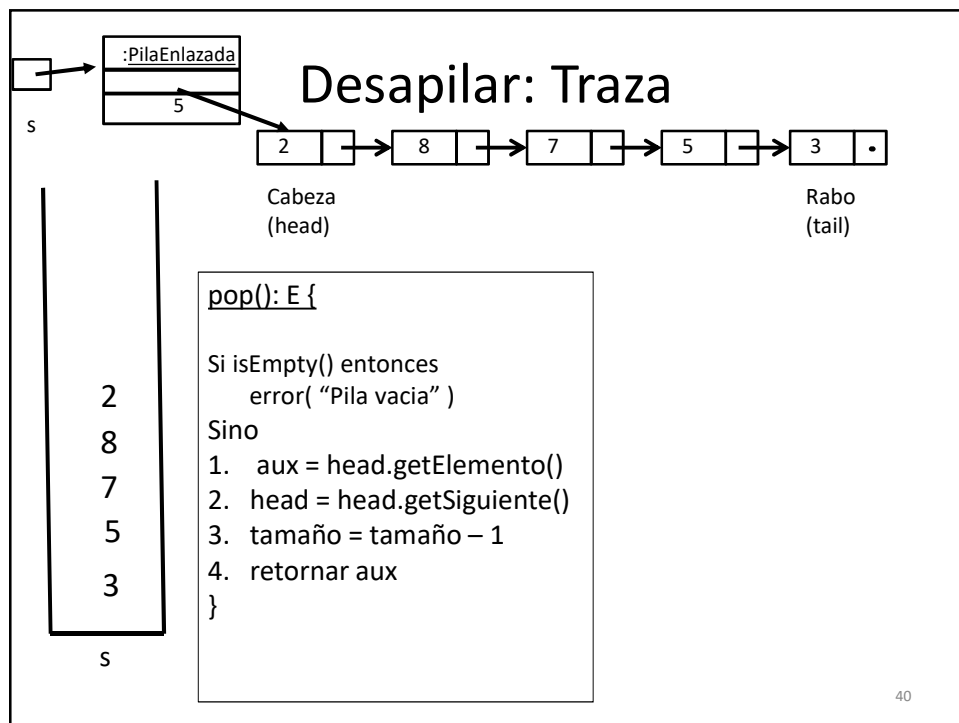
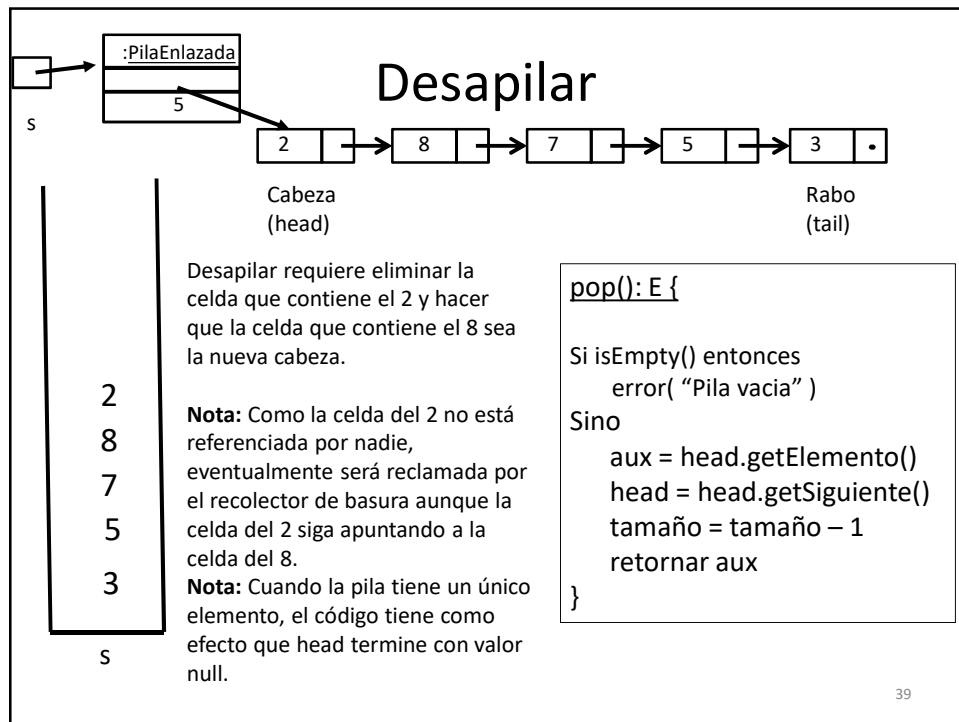
En este caso como la pila *s* está vacía el campo *head* contiene null:

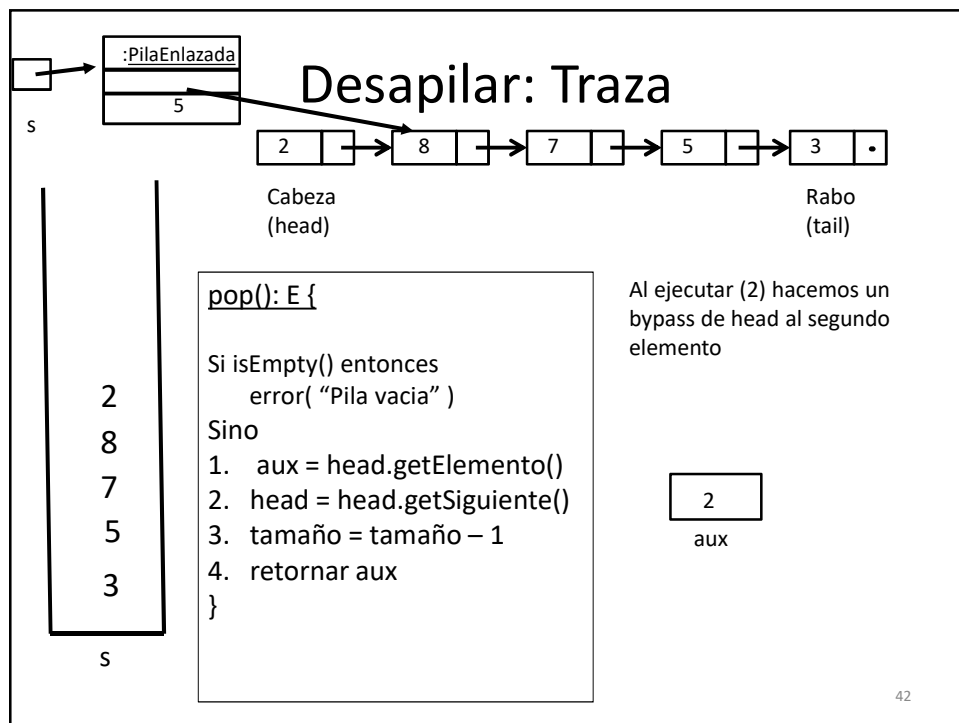
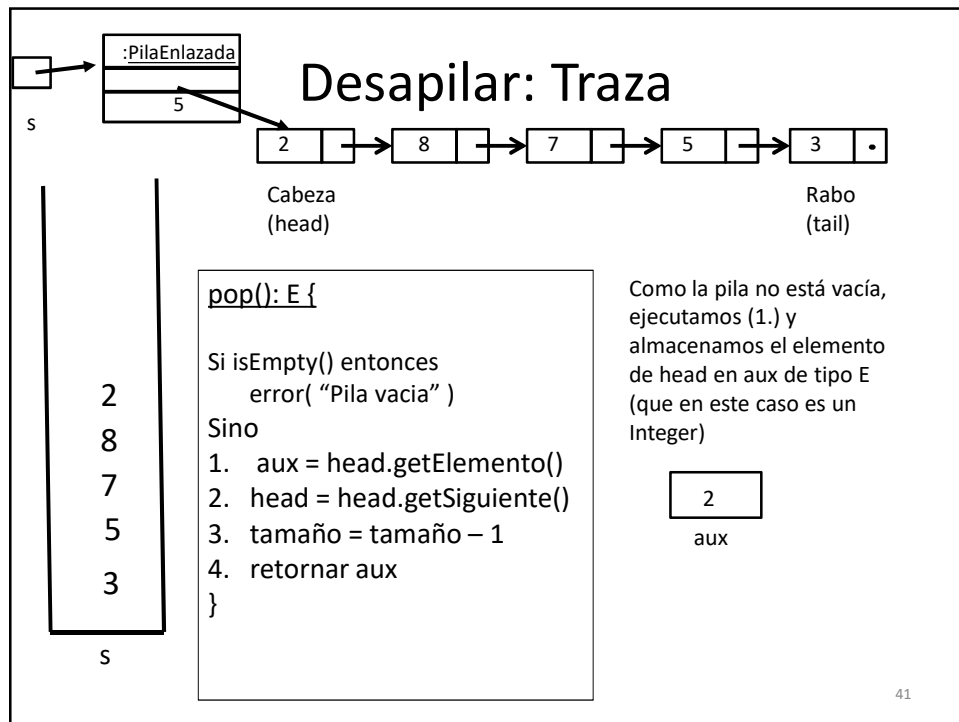


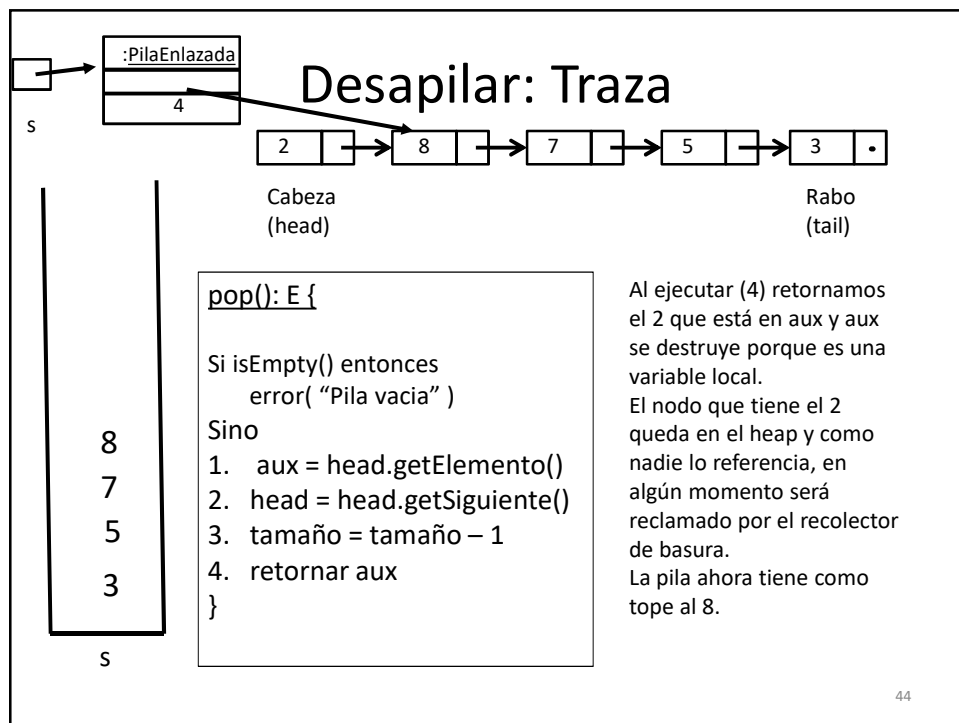
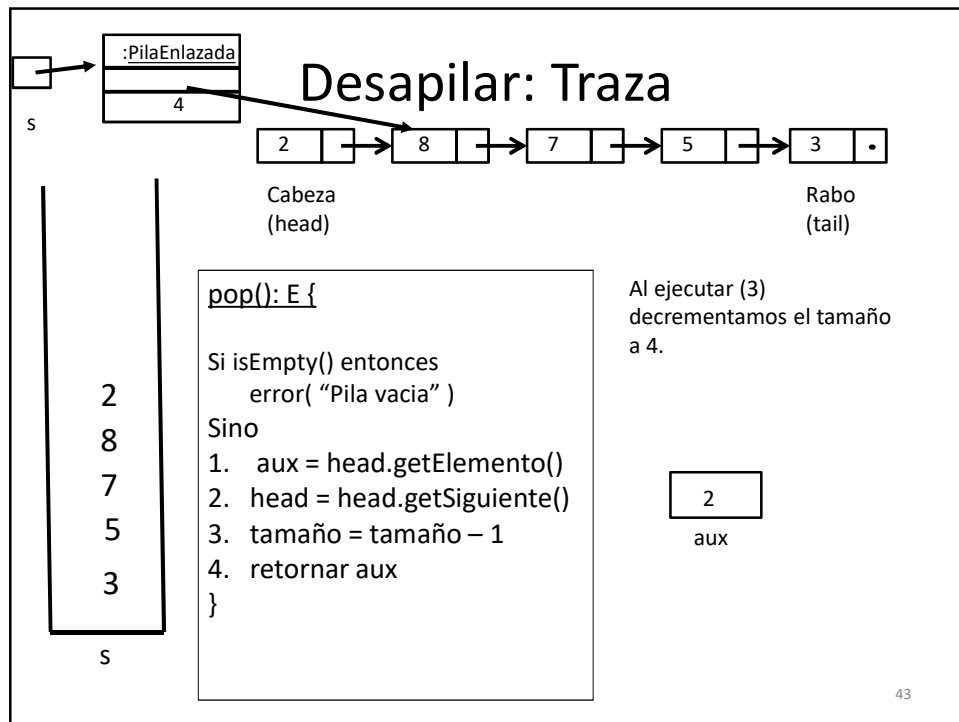
En este otro caso como la pila *s* no está vacía el campo *head* apunta al nodo tope (que contiene 5):



38







En Java el error se señala con una excepción

Debemos crear una excepción de programador llamada EmptyStackException que sea subclase de RuntimeException.

```
public E pop() {
    if isEmpty() {
        throw new EmptyStackException( "PilaEnlazada:pop:: Pila vacia" );
    } else {
        E aux = head.getElemento();
        head = head.getSiguiente();
        tamaño--;
        return aux;
    }
}
```

Estructuras de datos- Dr. Sergio A. Gómez

45

Refactorización con guarded clauses

```
public E pop() {
    if isEmpty() {
        throw new EmptyStackException( "PilaEnlazada:pop:: Pila vacia" );
    } else {
        E aux = head.getElemento();
        head = head.getSiguiente();
        tamaño--;
        return aux;
    }
}
```

Como throw termina la ejecución del método, se reescribe como:

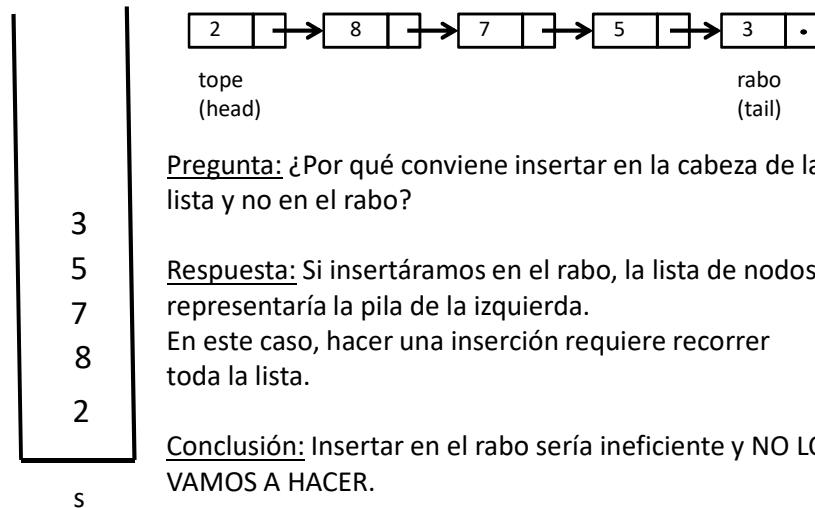
```
public E pop() {
    if isEmpty() throw new EmptyStackException( "PilaEnlazada:pop:: Pila vacia" );
    E aux = head.getElemento();
    head = head.getSiguiente();
    tamaño--;
    return aux;
}
```

Estructuras de datos- Dr. Sergio A. Gómez

46

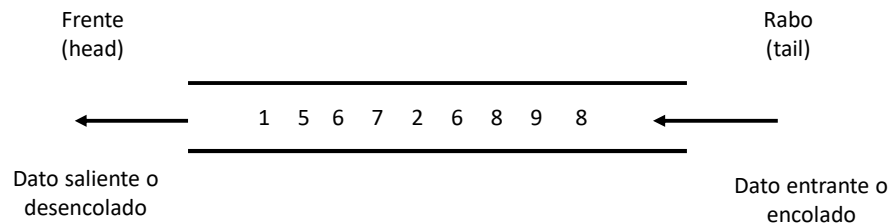
El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Comentarios sobre estrategia incorrecta de apilado



47

TDA Cola



Una *cola* (queue) es una colección con modelo de secuencia cuyos datos ingresan por un extremo llamado *rabo* (tail) y salen por otro extremo llamado *frente* (head).

Esta política de actualización se llama FIFO (por *first-in first-out*) o también FCFS (*first-come first-served*).

Estructuras de datos - Dr. Sergio A. Gómez

48

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.


```

public interface Queue<E> {
    // Inserta el elemento e al final de la cola
    public void enqueue(E item);

    // Elimina el elemento del frente de la cola y lo retorna.
    // Si la cola está vacía se produce un error.
    public E dequeue();

    // Retorna el elemento del frente de la cola.
    // Si la cola está vacía se produce un error.
    public E front();

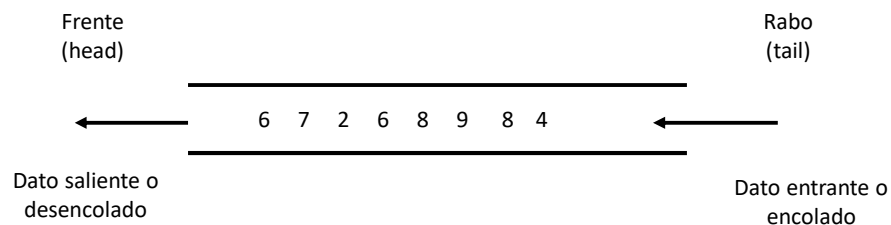
    // Retorna verdadero si la cola no tiene elementos
    // y falso en caso contrario
    public boolean isEmpty();

    // Retorna la cantidad de elementos de la cola.
    public int size();
}

```

49

TDA Cola



```

Queue<Integer> q = new ColaEnlazada<Integer>();
q.enqueue(1);
q.enqueue(5);
q.enqueue(6);
q.enqueue(7);
....
q.enqueue(8);
int elemento = q.dequeue(); // elemento vale 1 y se borra de la cola
q.dequeue();               // borro el 2 de la cola y no lo almaceno
q.enqueue(4);              // Puedo encolar cuando quiera

```

Estructuras de datos - Dr. Sergio A. Gómez

50

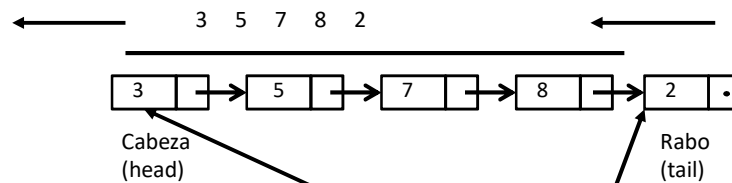
El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Implementaciones de colas

- 1) Con un arreglo circular:
 - 1) Requiere el tamaño inicial
 - 2) Requiere aritmética de módulo para funcionar eficientemente (para no para no hacer corrimientos al eliminar)
 - 3) Presenta complicaciones para agrandar el arreglo cuando se agota la capacidad
- 2) Con una estructura enlazada:
 - 1) No requiere tamaño inicial
 - 2) Requiere mantener la información de los dos extremos para implementar en forma eficiente
 - 3) La estructura es no acotada
- 3) En términos de una lista (lo dejamos pendiente hasta dar el TDA Lista)

51

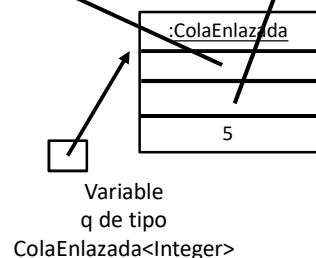
Cola: Implementación con nodos enlazados



```
public class ColaEnlazada<E>
    implements Queue<E> {
    protected Nodo<E> head, tail;
    protected int tamaño;

    ....
}
```

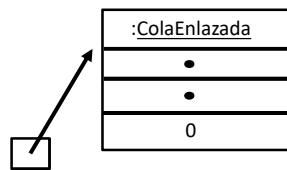
Nota: No va a haber límite en la cantidad de elementos que se pueden encolar.



52

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Cola enlazada: Constructor



q

```
Queue<Integer> q = new ColaEnlazada<Integer>();
```

```
public class ColaEnlazada<E>
    implements Queue<E> {
    protected Nodo<E> head, tail;
    protected int tamaño;

    ....
}
```

```
public ColaEnlazada() {
    head = null;
    tail = null;
    tamaño = 0;
}
```

53

Hacia la implementación de encolar y desencolar

Heurísticas a la hora de plantear implementación:

Plantear un caso cuando la estructura está vacía.

Plantear otro caso cuando la estructura tiene un único nodo.

Plantear otro caso cuando la estructura tiene 2 nodos o más.

Como tenemos 3 alternativas, la estructura de control a elegir será un if-then-else anidado.

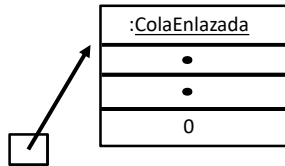
Luego podemos ver que hay casos que son iguales y se pueden refactorizar.

Luego podemos ver que hay sentencias comunes al principio o al final de los if-else y éstas se pueden factorizar arriba del if-else o debajo según corresponda.

En el caso de esta teoría mostramos la implementación de libro que ya tiene en cuenta estas cuestiones.

54

Cola: encolar en la cola vacía



q

Quiero ejecutar:

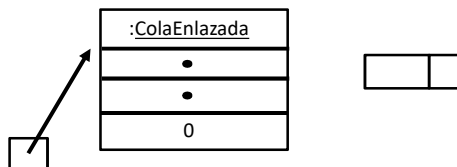
q.enqueue(3);

Veamos la traza:

```
public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();
2.  nodo.setElemento( item );
3.  nodo.setSiguiete( null );
4.  if (head == null )
5.      head = nodo;
6.  else
7.      tail.setSiguiete( nodo );
8.  tail = nodo;
9.  tamaño++;
}
```

55

Cola: encolar en la cola vacía



q

Quiero ejecutar:

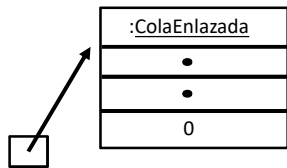
q.enqueue(3);

```
public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();

}
```

56

Cola: encolar en la cola vacía



q

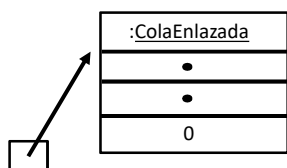
Quiero ejecutar:

q.enqueue(3);

```
public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();
2.  nodo.setElemento( item );
}
```

57

Cola: encolar en la cola vacía



q

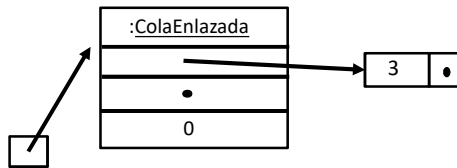
Quiero ejecutar:

q.enqueue(3);

```
public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();
2.  nodo.setElemento( item );
3.  nodo.setSiguiete( null );
}
```

58

Cola: encolar en la cola vacía

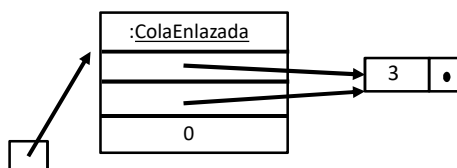


q
Quiero ejecutar:
q.enqueue(3);

```
public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();
2.  nodo.setElemento( item );
3.  nodo.setSiguiete( null );
4.  if ( head == null )
5.      head = nodo;
6.  else
7.      tail.setSiguiete( nodo );
}
```

59

Cola: encolar en la cola vacía

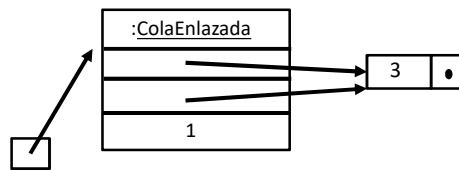


q
Quiero ejecutar:
q.enqueue(3);

```
public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();
2.  nodo.setElemento( item );
3.  nodo.setSiguiete( null );
4.  if ( head == null )
5.      head = nodo;
6.  else
7.      tail.setSiguiete( nodo );
8.  tail = nodo;
}
```

60

Cola: encolar en la cola vacía



q

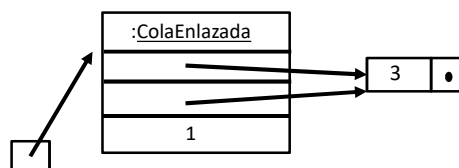
Quiero ejecutar:

q.enqueue(3);

```
public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();
2.  nodo.setElemento( item );
3.  nodo.setSiguiete( null );
4.  if ( head == null )
5.      head = nodo;
6.  else
7.      tail.setSiguiete( nodo );
8.  tail = nodo;
9.  tamaño++;
}
```

61

Cola: encolar en la cola no vacía



q

Quiero ejecutar:

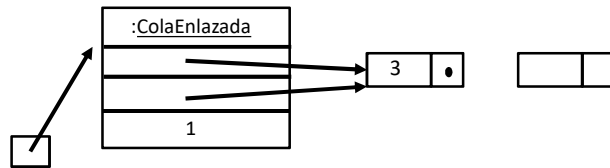
q.enqueue(5);

Veamos la traza:

```
public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();
2.  nodo.setElemento( item );
3.  nodo.setSiguiete( null );
4.  if ( head == null )
5.      head = nodo;
6.  else
7.      tail.setSiguiete( nodo );
8.  tail = nodo;
9.  tamaño++;
}
```

62

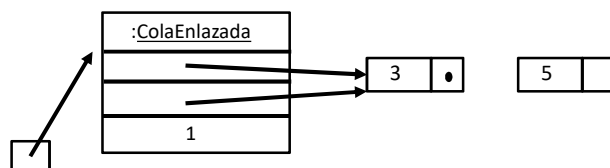
Cola: encolar en la cola no vacía



q
Quiero ejecutar:
q.enqueue(5);

```
public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();
}
63
```

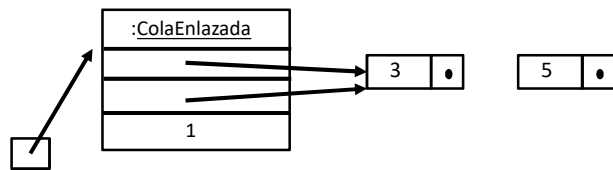
Cola: encolar en la cola no vacía



q
Quiero ejecutar:
q.enqueue(5);

```
public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();
2.  nodo.setElemento( item );
}
64
```


Cola: encolar en la cola no vacía



q

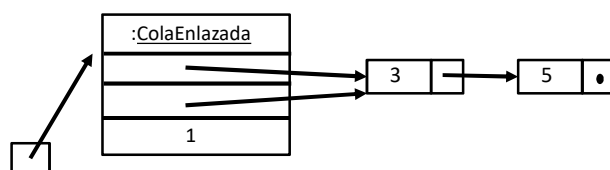
Quiero ejecutar:

q.enqueue(5);

```
public void enqueue( E item ) {
    1. Nodo<E> nodo = new Nodo<E>();
    2. nodo.setElemento( item );
    3. nodo.setSiguiete( null );
}
```

65

Cola: encolar en la cola no vacía



q

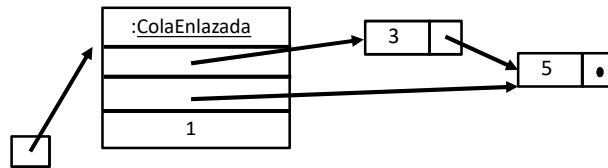
Quiero ejecutar:

q.enqueue(5);

```
public void enqueue( E item ) {
    1. Nodo<E> nodo = new Nodo<E>();
    2. nodo.setElemento( item );
    3. nodo.setSiguiete( null );
    4. if ( head == null )
    5.     head = nodo;
    6. else
    7.     tail.setSiguiete( nodo );
}
```

66

Cola: encolar en la cola no vacía



q

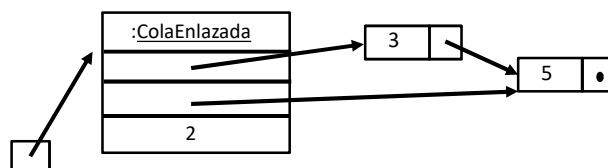
Quiero ejecutar:

q.enqueue(5);

```
public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();
2.  nodo.setElemento( item );
3.  nodo.setSiguiete( null );
4.  if ( head == null )
5.      head = nodo;
6.  else
7.      tail.setSiguiete( nodo );
8.  tail = nodo;
}
```

67

Cola: encolar en la cola no vacía



q

Quiero ejecutar:

q.enqueue(5);

```
public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();
2.  nodo.setElemento( item );
3.  nodo.setSiguiete( null );
4.  if ( head == null )
5.      head = nodo;
6.  else
7.      tail.setSiguiete( nodo );
8.  tail = nodo;
9.  tamaño++;
}
```

68

Cola: encolar en cola no vacía

Quiero ejecutar

`q.enqueue(4);`

Veamos la traza

```
public void enqueue( E item ) {
```

69

Cola: encolar en cola no vacía

Quiero ejecutar

`q.enqueue(4);`

Veamos la traza

```
public void enqueue( E item ) {
1. Nodo<E> nodo = new Nodo<E>();
2. nodo.setElemento( item );
3. nodo.setSiguiente( null );
}
```

70

Cola: encolar en cola no vacía

Quiero ejecutar
q.enqueue(4);
Veamos la traza

```

public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();
2.  nodo.setElemento( item );
3.  nodo.setSiguiente( null );
4.  if ( head == null )
5.      head = nodo;
6.  else
7.      tail.setSiguiente( nodo );
}

```

71

Cola: encolar en cola no vacía

Quiero ejecutar
q.enqueue(4);
Veamos la traza

```

public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();
2.  nodo.setElemento( item );
3.  nodo.setSiguiente( null );
4.  if ( head == null )
5.      head = nodo;
6.  else
7.      tail.setSiguiente( nodo );
8.  tail = nodo;
}

```

72

Cola: encolar en cola no vacía

Quiero ejecutar

`q.enqueue(4);`

Veamos la traza

```

public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();
2.  nodo.setElemento( item );
3.  nodo.setSiguiente( null );
4.  if ( head == null )
5.      head = nodo;
6.  else
7.      tail.setSiguiente( nodo );
8.  tail = nodo;
9.  tamaño++;
}

```

73

Implementación de isEmpty()

Caso cuando la cola está vacía:

Caso cuando la cola tiene elementos:

Entonces isEmpty() devuelve un booleano que se puede obtener de una de 2 formas:

1. Testear si tamaño es 0.
2. Testear si head es null.

Estructuras de Datos - Dr. Sergio A. Gómez

74

Cola: desencolar

```

public E dequeue() {
1. if( isEmpty() )
2.   throw new EmptyQueueException( "cola vacía" );

3. E tmp = head.getElemento();
4. head = head.getNext();
5. tamaño--;
6. if( head == null ) tail = null;
7. return tmp;
}

```

Quiero ejecutar
q.dequeue();
Veamos la traza.

75

Cola: desencolar

```

public E dequeue() {
1. if( isEmpty() )
2.   throw new EmptyQueueException( "cola vacía" );

}

```

Quiero ejecutar
q.dequeue();
Veamos la traza.

76

Cola: desencolar

```

public E dequeue() {
1. if( isEmpty() )
2.  throw new EmptyQueueException( "cola vacía" );

3. E tmp = head.getElemento();

}

```

Quiero ejecutar
q.dequeue();
Veamos la traza.

77

Cola: desencolar

```

public E dequeue() {
1. if( isEmpty() )
2.  throw new EmptyQueueException( "cola vacía" );

3. E tmp = head.getElemento();
4. head = head.getNext();

}

```

Quiero ejecutar
q.dequeue();
Veamos la traza.

78

Cola: desencolar

```

public E dequeue() {
1. if( isEmpty() )
2.  throw new EmptyQueueException( "cola vacía" );

3. E tmp = head.getElemento();
4. head = head.getNext();
5. tamaño --;

}

```

Quiero ejecutar
q.dequeue();
Veamos la traza.

79

Cola: desencolar

```

public E dequeue() {
1. if( isEmpty() )
2.  throw new EmptyQueueException( "cola vacía" );

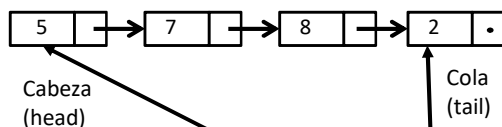
3. E tmp = head.getElemento();
4. head = head.getNext();
5. tamaño --;
6. if( head == null ) tail = null;
7. return tmp;
}

```

Se destruye la variable local tmp,
Se retorna el 3
El nodo del 3 queda en el heap como Objeto inalcanzable
Y lo reclama el recolector de basura.

80

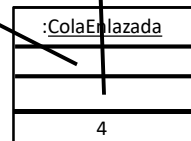
Cola: desencolar



```

public E dequeue() {
1. if( isEmpty() )
2.   throw new EmptyQueueException( "cola vacía" );

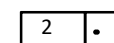
3. E tmp = head.getElemento();
4. head = head.getNext();
5. tamaño--;
6. if( head == null ) tail = null;
7. return tmp;
}
  
```



Esta es la cola ahora.

81

Cola: desencolar cuando la q.size() = 1



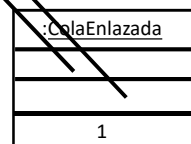
Cabeza
(head)

Cola
(tail)

```

public E dequeue() {
1. if( isEmpty() )
2.   throw new EmptyQueueException( "cola vacía" );

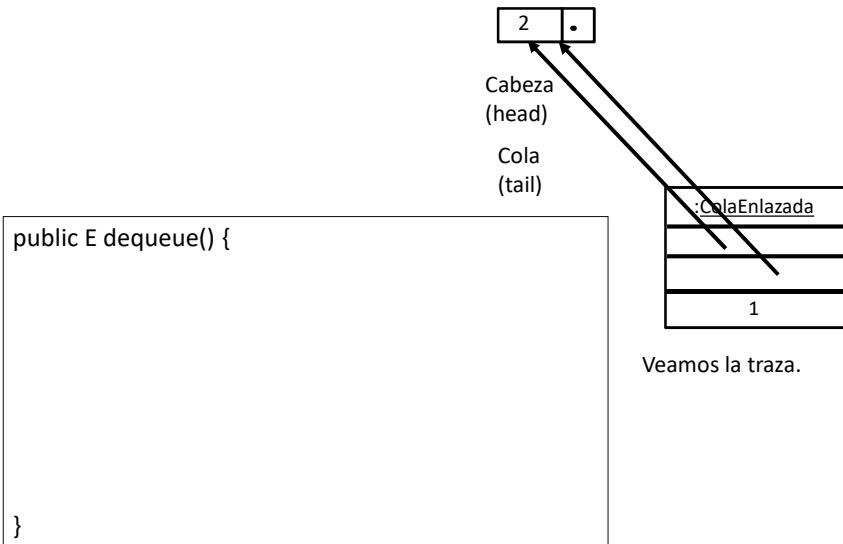
3. E tmp = head.getElemento();
4. head = head.getNext();
5. tamaño--;
6. if( head == null ) tail = null;
7. return tmp;
}
  
```



Ahora head y tail apuntan al mismo nodo.

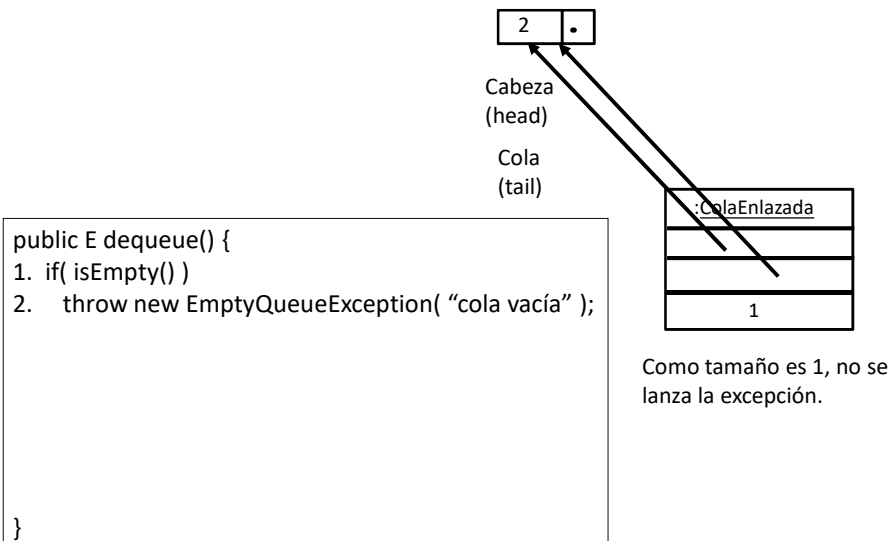
82

Cola: desencolar cuando la $q.size() = 1$



83

Cola: desencolar cuando la $q.size() = 1$



84

Cola: desencolar cuando la q.size() = 1

2
tmp

2 .

Cabeza
(head)

Cola
(tail)

```
public E dequeue() {
1. If ( isEmpty() )
2.  throw new EmptyQueueException( "cola vacía" );

3. E tmp = head.getElemento();

}
```



Se crea una variable
local tmp y se asigna 2

85

Cola: desencolar cuando la q.size() = 1

2
tmp

2 .

Cabeza
(head)

Cola
(tail)

```
public E dequeue() {
1. if ( isEmpty() )
2.  throw new EmptyQueueException( "cola vacía" );

3. E tmp = head.getElemento();
4. head = head.getNext();

}
```



Como el siguiente de
head es null, head toma
el valor null

86

Cola: desencolar cuando la q.size() = 1

2
tmp

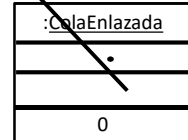
2 .

Cabeza
(head)
Cola
(tail)

```
public E dequeue() {
1. if( isEmpty() )
2.  throw new EmptyQueueException( "cola vacía" );

3. E tmp = head.getElemento();
4. head = head.getNext();
5. tamaño --;

}
```



Decrementamos
tamaño a 0

87

Cola: desencolar cuando la q.size() = 1

2
tmp

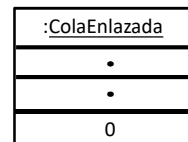
2 .

Cabeza
(head)
Cola
(tail)

```
public E dequeue() {
1. if( isEmpty() )
2.  throw new EmptyQueueException( "cola vacía" );

3. E tmp = head.getElemento();
4. head = head.getNext();
5. tamaño --;
6. if( head == null ) tail = null;

}
```



Como tamaño es 0,
seteamos tail a null

88

Cola: desencolar cuando la $q.size() = 1$

2	.
---	---

→2

```
public E dequeue() {
1. if( isEmpty() )
2.   throw new EmptyQueueException( "cola vacía" );

3. E tmp = head.getElemento();
4. head = head.getNext();
5. tamaño --;
6. if( head == null ) tail = null;
7. return tmp;
}
```

:ColaEnlazada	
	.
	.
	0

Se retorna el 2, se destruye la variable local tmp, el nodo queda inalcanzable en el heap y se lo lleva el recolector de basura.

89

Cola: desencolar cuando $q.size() = 0$ y $q.isEmpty() = true$

Ahora la cola está vacía.
¿Qué ocurrirá ahora con la ejecución de dequeue()?

:ColaEnlazada	
	.
	.
	0

```
public E dequeue() {

}
}
```

90

Cola: desencolar cuando $q.size() = 0$ y $q.isEmpty() = true$

Como tamaño es 0, dequeue() termina con error.
No se devuelve nada y se lanza una excepción de tipo EmptyQueueException.

:ColaEnlazada
•
•
0

```
public E dequeue() {
1. if( isEmpty() )
2.  throw new EmptyQueueException( "cola vacía" );

}
```

91

Ejemplo: Insertar los elementos 1, 2, 3, 4 en una cola y luego mostrar todos los elementos de la cola.

```
public class App
{
    public static void main( String [] args ) {
        try {
            Queue<Integer> q = new ColaEnlazada<Integer>();

            for( int i=1; i<=4; i++)
                q.enqueue( i );
            while( !q.isEmpty() )
                System.out.println( q.dequeue() );
        } catch( EmptyQueueException e ) {
            e.printStackTrace();
        }
    }
}
```

92

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
"Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Bibliografía

- Goodrich & Tamassia, Data Structures and Algorithms in Java, 4th edition, John Wiley & Sons, 2006, Capítulo 5
- Se puede profundizar la parte de nodos enlazados en la Sección 3.2 del libro

93